

Unit-1: Working with HTML5 and CSS

1.1 Concepts of CSS

CSS is used to define how the HTML elements are displayed in the browser. CSS is the language we use to style an HTML document.

What is CSS?

- CSS stands for Cascading Style Sheets
- CSS describes how HTML elements are to be displayed on screen, paper, or in other media.
- CSS saves a lot of work. It can control the layout of multiple web pages all at once.
- External style sheets are stored in CSS files. (file extension = .css)

What you need before learning CSS?

Before learning CSS, you should have basic knowledge about HTML.

Why Use CSS?

CSS is used to define styles for your web pages, including the design, layout and variations in display for different devices and screen sizes.

CSS Example:

```
body {  
    background-color: lightblue;  
}  
h1 {  
    color: white;  
    text-align: center;  
}  
p {  
    font-family: verdana;  
    font-size: 20px;  
}
```

Advantages of CSS

CSS saves time - You can write CSS once and then reuse same sheet in multiple HTML pages. You can define a style for each HTML element and apply it to as many Web pages as you want.

Pages load faster - If you are using CSS, you do not need to write HTML tag attributes every time. Just write one CSS rule of a tag and apply it to all the occurrences of that tag. So less code means faster download times.

Easy maintenance - To make a global change, simply change the style, and all elements in all the web pages will be updated automatically.

Superior styles to HTML - CSS has a much wider array of attributes than HTML, so you can give a far better look to your HTML page in comparison to HTML attributes.

Multiple Device Compatibility - Style sheets allow content to be optimized for more than one type of device. By using the same HTML document, different versions of a website can be presented for handheld devices such as PDAs and cell phones or for printing.

Global web standards - Now HTML attributes are being deprecated and it is being recommended to use CSS. So its a good idea to start using CSS in all the HTML pages to make them compatible to future browsers.

CSS Syntax

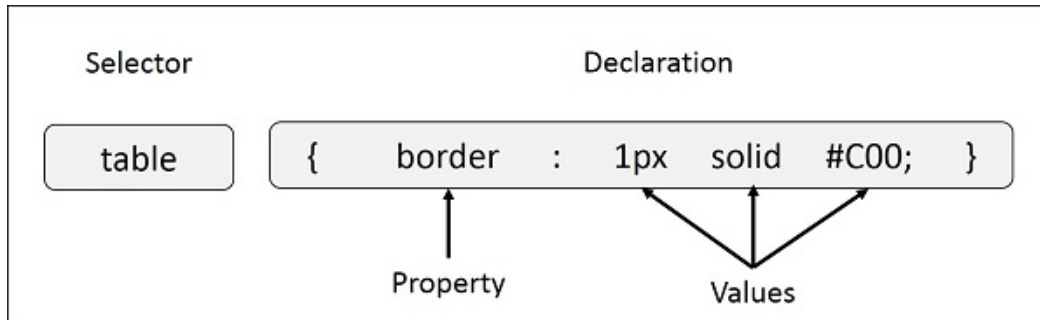
A CSS comprises of style rules that are interpreted by the browser and then applied to the corresponding elements in your document. A style rule is made of three parts –

Selector – A selector is an HTML tag at which a style will be applied. This could be any tag like <h1> or <table> etc. there are other selectors also like class, ID, attribute etc.

Property – A property is a type of attribute of HTML tag. Put simply, all the HTML attributes are converted into CSS properties. They could be color, border etc.

Value – Values are assigned to properties. For example, color property can have value either red or #F1F1F1 etc.

The Syntax is: **selector {property: value}**



For example, `table{ border :1px solid #C00; }`.

Multiple CSS declarations are separated with semicolons, and declaration blocks are surrounded by curly braces. For example,

```
p {  
  color: red;  
  text-align: center;  
}
```

- p is a selector in CSS (it points to the HTML element you want to style: <p>).
- color is a property, and red is the property value
- text-align is a property, and center is the property value

CSS Selectors

The Type Selectors

This is the same selector we have seen above. Again, one more example to give a color to all level 1 headings:

```
h1 {  
  color: #36CFFF;  
}
```

The Universal Selectors

Rather than selecting elements of a specific type, the universal selector quite simply matches the name of any element type –

```
* {
```

```
color: #0000FF;  
}
```

This rule renders the content of every element in our document in black.

The Descendant Selectors

Suppose you want to apply a style rule to a particular element only when it lies inside a particular element. As given in the following example, style rule will apply to `<em>` (emphasize) element only when it lies inside `<ul>` tag.

```
ul em {  
    color: #00FF00;  
}
```

The Class Selectors

You can define style rules based on the class attribute of the elements. All the elements having that class will be formatted according to the defined rule.

```
.grey {  
    color: #707070;  
}
```

This rule renders the content in black for every element with class attribute set to black in our document. You can make it a bit more particular. For example:

```
h1.black {  
    color: #000000; }
```

This rule renders the content in black for only `<h1>` elements with class attribute set to black.

You can apply more than one class selectors to given element. Consider the following example:

```
<p class="center bold">  
    This para will be styled by two classes center and bold.  
</p>
```

The ID Selectors

You can define style rules based on the id attribute of the elements. All the elements having that id will be formatted according to the defined rule.

```
#black {  
    color: #000000;  
}
```

This rule renders the content in black for every element with id attribute set to black in our document. You can make it a bit more particular. For example –

```
h1#black {  
    color: #000000;  
}
```

This rule renders the content in black for only <h1> elements with id attribute set to black.

The true power of id selectors is when they are used as the foundation for descendant selectors, For example:

```
#black h2 {  
    color: #000000;  
}
```

In this example all level 2 headings will be displayed in black color when those headings will lie within tags having id attribute set to black.

The Child Selectors

You have seen the descendant selectors. There is one more type of selector, which is very similar to descendants but have different functionality. Consider the following example –

```
body > p {  
    color: #000000;  
}
```

This rule will render all the paragraphs in black if they are direct child of <body> element. Other paragraphs put inside other elements like <div> or <td> would not have any effect of this rule.

The Attribute Selectors

You can also apply styles to HTML elements with particular attributes. The style rule below will match all the input elements having a type attribute with a value of text –

```
input[type = "text"]{  
    color: #000000; }
```

The advantage to this method is that the <input type = "submit" /> element is unaffected, and the color applied only to the desired text fields.

There are following rules applied to attribute selector.

- **p[lang]** - Selects all paragraph elements with a *lang* attribute.
- **p[lang="fr"]** - Selects all paragraph elements whose *lang* attribute has a value of exactly "fr".
- **p[lang~="fr"]** - Selects all paragraph elements whose *lang* attribute contains the word "fr".
- **p[lang|="en"]** - Selects all paragraph elements whose *lang* attribute contains values that are exactly "en", or begin with "en-".

Multiple Style Rules

You may need to define multiple style rules for a single element. You can define these rules to combine multiple properties and corresponding values into a single block as defined in the following example –

```
h1 {  
  color: #36C;  
  font-weight: normal;  
  letter-spacing: .4em;  
  margin-bottom: 1em;  
  text-transform: lowercase;  
}
```

Here all the property and value pairs are separated by a semi-colon. You can keep them in a single line or multiple lines. For better readability we keep them into separate lines.

Grouping Selectors

You can apply a style to many selectors if you like. Just separate the selectors with a comma, as given in the following example –

```
h1, h2, h3 {  
  color: #36C;  
  font-weight: normal;  
  letter-spacing: .4em;  
  margin-bottom: 1em;  
  text-transform: lowercase; }
```

This define style rule will be applicable to h1, h2 and h3 element as well. The order of the list is irrelevant. All the elements in the selector will have the corresponding declarations applied to them. You can combine the various class selectors together as shown below

```
#content, #footer, #supplement {  
  position: absolute;  
  left: 510px;  
  width: 200px; }
```

1.1.1 Adding CSS (Three Ways to Insert CSS / Types of CSS)

There are three ways of inserting a style sheet:

- External CSS
- Internal CSS
- Inline CSS

External CSS

With an external style sheet, you can change the look of an entire website by changing just one file.

Each HTML page must include a reference to the external style sheet file inside the <link> element, inside the head section.

An external style sheet can be written in any text editor, and must be saved with a .css extension. The external .css file should not contain any HTML tags.

Example:

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" href="mystyle.css">
</head>
<body>
<h1>This is a heading</h1>
<p>This is a paragraph.</p>
</body>
</html>

- - - - - "mystyle.css"
body {
  background-color: lightblue;
}
h1 {
  color: navy;
  margin-left: 20px;
}
```

Internal CSS

An internal style sheet may be used if one single HTML page has a unique style. The internal style is defined inside the <style> element, inside the head section.

Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
  background-color: linen;
}

h1 {
  color: maroon;
  margin-left: 40px;
}
</style>
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

Inline CSS

An inline style may be used to apply a unique style for a single element.

To use inline styles, add the style attribute to the relevant element. The style attribute can contain any CSS property.

Example:

```
<!DOCTYPE html>
<html>
```

```
<body>
```

```
<h1 style="color:blue; text-align:center;">This is a heading</h1>
```

```
<p style="color:red;">This is a paragraph.</p>
```

```
</body> </html>
```

CSS Comments

Comments are used to explain the code, and may help when you edit the source code at a later date. Comments are ignored by browsers.

A CSS comment starts with `/*` and ends with `*/`:

Example

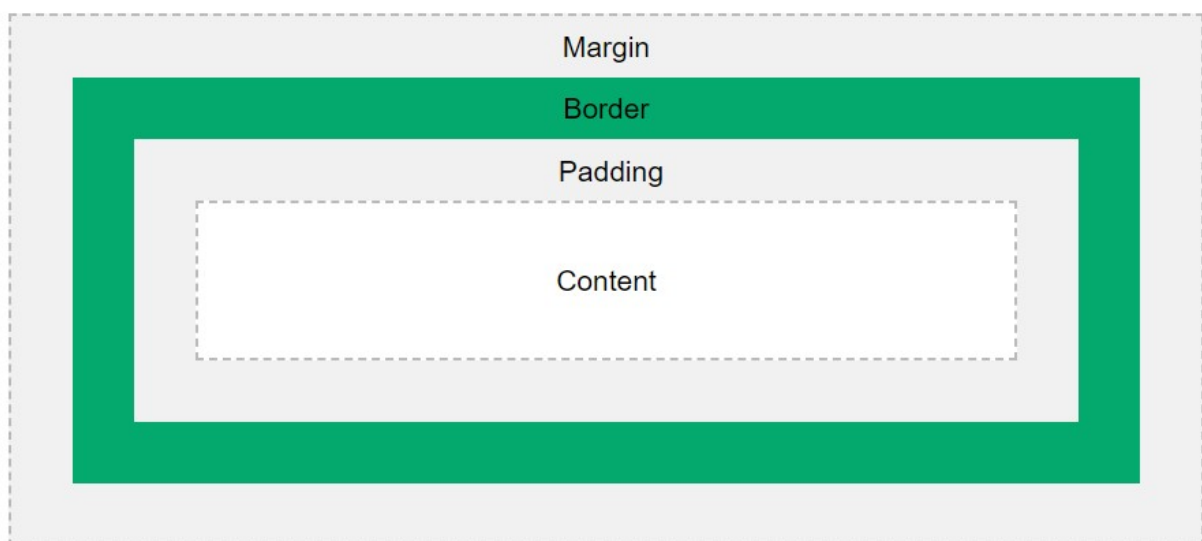
```
/* This is a single-line comment */
```

```
p {  
  color: red;      /* set the color to red */  
}
```

CSS Box Model

In CSS, the term "box model" is used when talking about design and layout.

The CSS box model is essentially a box that wraps around every HTML element. It consists of: margins, borders, padding, and the actual content. The image below illustrates the box model.



Explanation of the different parts:

Content - The content of the box, where text and images appear

Padding - Clears an area around the content. The padding is transparent

Border - A border that goes around the padding and content

Margin - Clears an area outside the border. The margin is transparent

The box model allows us to add a border around elements, and to define space between elements.

Example:

```
div {  
  width: 300px;  
  border: 15px solid green;  
  padding: 50px;  
  margin: 20px;  
}
```

1.1.2 HTML links and attribute: `_self`, `_blank`, `_parent`, `_top` (Anchor tag `<a>...</a>`)

Websites contain different types of links that take you directly to other pages or allow navigating to a particular part of the page. The links in HTML are called hyperlinks. They are defined using the `<a>` tag. It is also known as HTML anchor tag.

The "href" attribute is the most important attribute of the HTML `a` tag, and which links to destination page or URL. The href attribute is used to define the address of the file to be linked. In other words, it points out the destination page. The `<a>` tag comes in pairs, the opening `<a>` tells where the link should start and the closing `</a>` indicates where the link ends. Hyperlinks are applied to a phrase, a word, an image or any HTML element.

The default color of links in HTML is:

- ☐ unvisited links: underlined and blue
- ☐ visited links: underlined and purple
- ☐ active links: underlined and red

This is default style of links, but you can remove underline or change the color of the links using CSS styles. The target attribute specifies the default

browsing context to load the URL into. Only to be used when the "href" attribute is present.

The **target attribute** is used to specify the location where linked document is opened. Following are the possible values:

- ☐ **_blank**: Opens the linked document in a new window or tab.
- ☐ **_self**: Opens the linked document in the same frame.
- ☐ **_parent**: Opens the linked document in the parent frame.
- ☐ **_top**: Opens the linked document in the full body of the window.

Note: **_parent** and **_top** are useful when you have HTML page structure where there is a window within another window (can be done using `iframe`).

1.1.3 Absolute URL and Relative URL in <href>

The href attribute specifies the location (URL) of the external resource (most often a style sheet file).

Possible values:

An absolute URL - points to another web site (like
`href="http://www.example.com/theme.css"`)

A relative URL - points to a file within a web site (like
`href="/themes/theme.css"`)

1.1.4 tag and its attributes (src, alt, style, width, height)

The tag is used to embed an image in an HTML page.

Images are not technically inserted into a web page; images are linked to web pages. The tag creates a holding space for the referenced image.

The tag has two required attributes:

src - Specifies the path to the image

alt - Specifies an alternate text for the image, if the image for some reason cannot be displayed.

style – Specifies CSS styling for the element.

width – Specifies width of the image.

height – Specifies height of the image.

Example:

```

```

To link an image to another document, we nest the tag inside an <a> tag.

1.2 HTML forms

HTML Forms are required, when you want to collect some data from the site visitor. For example, during user registration you would like to collect information such as name, email address, credit card, etc.

There are various form elements available like text fields, textarea fields, drop-down menus, radio buttons, checkboxes, etc.

The HTML <form> tag is used to create an HTML form and it has following syntax:

```
<form action = "Script URL" method = "GET|POST">  
    form elements like input, textarea etc.  
</form>
```

→ Form Attributes

Action: Specifies URL of backend script file that is ready to process the data passed from the form.

Method: Method to be used to upload data. The most frequently used are GET and POST methods.

Target: Specify the target window or frame where the result of the script will be displayed. It takes values like _blank, _self, _parent etc.

Nonvalidate: The novalidate attribute is a boolean attribute. When present, it specifies that the form-data (input) should not be validated when submitted.

Autocomplete: The autocomplete attribute specifies whether a form should have autocomplete on or off. When autocomplete is on, the browser

automatically complete values based on values that the user has entered before. It has two values:

- On: Default. The browser will automatically complete values based on values that the user has entered before
- Off: The user must enter a value into each field for every use. The browser does not automatically complete entries.

Example:

```
<form action="php/action_page.php" novalidate autocomplete="on">  
  <label for="email">Enter your email:</label>  
  <input type="email" id="email" name="email"><br><br>  
  <input type="submit">  
</form>
```

→ **Form Elements**

Text Input Controls

There are three types:

Single-line text input controls – This control is used for items that require only one line of user input, such as search boxes or names. They are created using HTML <input> tag.

Password input controls – This is also a single-line text input but it masks the character as soon as a user enters it. They are also created using HTML <input> tag.

Multi-line text input controls – This is used when the user is required to give details that may be longer than a single sentence. Multi-line input controls are created using HTML <textarea> tag. Attributes rows and cols are used to specify size of the text area.

<input> tag attributes

Type: Indicates the type of input control and for text input control it will be set to **text**.

Name: Used to give a name to the control which is sent to the server to be recognized and get the value.

Value: This can be used to provide an initial value inside the control.

Size: Allows specifying the width of the text-input control in terms of characters.

Maxlength: Allows to specify the maximum number of characters a user can enter into the text box.

Password input controls

This is also a single-line text input but it masks the character as soon as a user enters it. They are also created using HTML `<input>` tag but type attribute is set to **password**.

Example code:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Text Input Control</title>
  </head>
  <body>
    <form >
      First name: <input type = "text" name = "first_name" />
      <br>
      Last name: <input type = "text" name = "last_name" />
      <br>
      User ID : <input type = "text" name = "user_id" />
      <br>
      Password: <input type = "password" name = "password" />
    </form>
  </body>
</html>
```

Multiple-Line Text Input Controls

This is used when the user is required to give details that may be longer than a single sentence. Multi-line input controls are created using HTML `<textarea>` tag.

Additional attributes:

Rows: Indicates the number of rows of text area box.

Cols: Indicates the number of columns of text area box

Example:

```
<form>
    Description : <br/>
    <textarea rows = "5" cols = "50" name = "description">
        Enter description here...
    </textarea>
</form>
```

Checkbox Control

Checkboxes are used when more than one option is required to be selected. They are also created using HTML `<input>` tag but type attribute is set to checkbox.

Attributes

Type: Indicates the type of input control and for checkbox input control it will be set to checkbox.

Name: Used to give a name to the control which is sent to the server to be recognized and get the value.

Value: The value that will be used if the checkbox is selected.

Checked: Set to checked if you want to select it by default.

Example:

```
<form>
    <input type = "checkbox" name = "maths" value = "on"> Maths
    <input type = "checkbox" name = "physics" value = "on"> Physics
</form>
```

Radio Button Control

Radio buttons are used when out of many options, just one option is required to be selected. They are also created using HTML `<input>` tag but type attribute is set to radio.

Attributes of radio button control are same as checkbox control.

Example:

```
<form>
    <input type = "radio" name = "subject" value = "maths"> Maths
```

```
        <input type = "radio" name = "subject" value = "physics">
Physics
</form>
```

Select Box (drop down box) Control

A select box, also called drop down box which provides option to list down various options in the form of drop down list, from where a user can select one or more options.

<select> attributes:

Name: Used to give a name to the control which is sent to the server to be recognized and get the value.

Size: This can be used to present a scrolling list box.

Multiple: If set to "multiple" then allows a user to select multiple items from the menu.

<option> tag is used to add options to the drop down box.

<option> attributes:

Value: The value that will be used if an option in the select box is selected.

Selected: Specifies that this option should be the initially selected value when the page loads.

```
<form>
    <select name = "dropdown">
        <option value = "Maths" selected> Maths </option>
        <option value = "Physics"> Physics </option>
        <option value = "Chemistry"> Physics </option>
    </select>
</form>
```

File Upload Box

If you want to allow a user to upload a file to your web site, you will need to use a file upload box, also known as a file select box. This is also created using the <input> element but type attribute is set to file.

attributes:

name: Used to give a name to the control which is sent to the server to be recognized and get the value.

Accept: Specifies the types of files that the server accepts.

```
<form>
    <input type = "file" name = "fileupload" accept = "image/*" />
</form>
```

Button Controls

There are various ways in HTML to create clickable buttons. You can also create a clickable button using `<input>` tag by setting its type attribute to button. The type attribute can take the following values:

type="submit": This creates a button that automatically submits a form.

type="reset": This creates a button that automatically resets form controls to their initial values.

type="button": This creates a button that is used to trigger a client-side script when the user clicks that button.

type="image": This creates a clickable button but we can use an image as background of the button.

Example:

```
<form>
    <input type = "submit" name = "submit" value = "Submit" />
    <input type = "reset" name = "reset" value = "Reset" />
    <input type = "button" name = "ok" value = "OK" />
    <input type = "image" name = "imgbutton" src="logo.png" />
</form>
```

To execute a JavaScript when a button is clicked, we can use **"onclick"** attribute to call a java script function.

Example:

```
<body>
<button onclick="myFunction()">Click me</button>
<p id="demo"></p>
<script>
function myFunction() {
    document.write("hello world");
}
</script>
</body>
```

HTML `<input>` list Attribute with datalist

The list attribute refers to a <datalist> element that contains pre-defined options for an <input> element.

Syntax: <input list="datalist-id">

Using <datalist> and <option> tags we can create a list. And then we can link this list to <input list=""> element to generate a drop down list. See following code for more understanding.

```
<input list="browsers">

<datalist id="browsers">
  <option value="Internet Explorer">
  <option value="Firefox">
  <option value="Google Chrome">
  <option value="Opera">
  <option value="Safari">
</datalist>
```

1.2.2 Media: Video, Audio

HTML <audio> Tag

The <audio> tag is used to embed sound content in a document, such as music or other audio streams.

The <audio> tag contains one or more <source> tags with different audio sources. The browser will choose the first source it supports.

The text between the <audio> and </audio> tags will only be displayed in browsers that do not support the <audio> element.

There are three supported audio formats in HTML: MP3, WAV, and OGG.

Attributes:

Autoplay: Specifies that the audio will start playing as soon as it is ready

Controls: Specifies that audio controls should be displayed (such as a play/pause button etc)

Loop: Specifies that the audio will start over again, every time it is finished

Muted: Specifies that the audio output should be muted

Src: Specifies the URL of the audio file

```
<audio controls>
  <source src="horse.ogg" type="audio/ogg">
  <source src="horse.mp3" type="audio/mpeg">
```

Your browser does not support the audio tag.

</audio>

HTML <video> Tag

The controls attribute adds video controls, like play, pause, and volume.

It is a good idea to always include width and height attributes. If height and width are not set, the page might flicker while the video loads.

The <source> element allows you to specify alternative video files which the browser may choose from. The browser will use the first recognized format.

The text between the <video> and </video> tags will only be displayed in browsers that do not support the <video> element.

<video> Attributes:

Height: specifies height for the video frame

Width: specifies width for the video frame

Controls: Specifies that video controls should be displayed (such as a play/pause button etc)

Autoplay: Specifies that video should start automatically.

Muted: specifies that the video should be muted.

<source> attributes:

src: specifies the video file path or URL.

Type: specifies the file format of the video file.

Example:

```
<video width="320" height="240" controls autoplay>
  <source src="movie.mp4" type="video/mp4">
</video>
```

Differentiate between inline and block HTML tags.

Every HTML element has a default display value, depending on what type of element it is.

There are two display values: block and inline.

- A block-level element always starts on a new line. An inline element continues on the same line.

- A block-level element always takes up the full width available from left to right. An inline element uses the required width on the same line.
- A block level element has a top and a bottom margin, whereas an inline element does not.